

Safe-Her

Women's Personal Safety Application — Full Project Audit

Date: 26 September 2026

Stack: Expo SDK 57 / React Native 0.86 / Expo Router

Backend: Python FastAPI (safety-engine)

Files reviewed: 52

Method: Static read + live runtime verification

Executive Summary — 2 Critical Defects Found

The architecture, component design and feature surface of this project are genuinely strong. There are **no TODO, FIXME or "coming soon" markers anywhere**, the SOS flow is honest about what it does, and the background-location implementation is textbook-correct.

However, two defects prevent the product's headline features from working. A missing import in the safety API client means **the safety score, safe-hub data and route planning never reach the backend** — the app silently substitutes invented numbers. A mismatched environment-variable name means **report submission and journey saving cannot reach the backend on a physical device**.

Both defects are silent: the UI renders plausible output rather than failing loudly, which is why they survived. **Both are fixable in roughly two lines of code.**

1. Scope & Methodology

How this audit was performed

VERIFIED

Every source file in the repository was read in full, top to bottom:

- **13 screens** — `app/_layout.jsx`, `+not-found`, `auth`, `gov`, `edit-profile`, `emergency-contacts`, `onboarding`, and all 5 tab screens
- **16 lib modules** — `auth`, `theme`, `storage`, `onboarding`, `sos`, `location`, `safetyApi`, `config`, `journeyApi`, `journey`, `preferences`, `notifications`, `localReports`, `emergencyContacts`, `emergencyMessage`, `alertSharing`, `backgroundLocationTask`
- **8 components** — `ui`, `safetyScore`, `SOSActiveScreen`, `SOSConfirmationModal`, `NavigationMap` (+ `.web`), `RouteMap` (+ `.web`)
- **15 Python files** — the entire `safety-engine` backend
- **Build & config** — `package.json`, `app.json`, `eas.json`, `tsconfig`, `.env`, `.gitignore`, `.easignore`, Android prebuild (37 tracked files), `manifest`

The two critical findings were additionally **confirmed empirically** by running the application and capturing network telemetry and the browser console. No project file was modified during this audit.

2. Critical Defects P0

2.1 Safety API client calls `apiFetch()` without importing it

CRITICAL

The module imports only from `./location`:

```
// lib/safetyApi.js:1 - the ONLY import in the file
import { haversineDistance, estimateTravelTime } from './location';
```

Yet `apiFetch` is called un-imported at three sites:

LINE	CALL	RUNTIME EFFECT
35	<code>apiFetch('/safety/analyze', ...)</code>	Not <code>AbortError</code> → rethrows
94	<code>apiFetch('/safety/spot', ...)</code>	Swallowed → hardcoded 68/82 fallback
146	<code>apiFetch('/safety/hubs', ...)</code>	Swallowed → empty + <code>_offline: true</code>

Empirical confirmation. With the app running and signed in, the Home screen actively invokes `fetchSpotSafety` and `fetchSafeHubs`. Network telemetry captured every backend request made by the entire session:

```
backendRequests: [ "/reports" ]

/safety/spot    - never requested
/safety/hubs    - never requested
/safety/analyze - never requested
```

Only `/reports` appears, because `lib/localReports.js` correctly imports `apiFetch`. The three safety endpoints are **never contacted**.

User-visible consequences:

- **Home safety score** — always the time-based estimate (68 day / 82 night), never real analysis
- **"Safety around you" hubs card** — always empty
- **Routes screen** — route planning fails and displays the raw string `apiFetch is not defined` to the user instead of a route

Fix: add `import { apiFetch } from './config';` to line 1, and delete the dead `SAFETY_API_BASE` constant (lines 19–23, never referenced).

2.2 Environment-variable name mismatch disables the device build

CRITICAL

Two different variable names are in use for the same setting:

NAME	DECLARED IN	STATUS
EXPO_PUBLIC_SAFETY_API	lib/config.js:10	Never set anywhere
EXPO_PUBLIC_SAFETY_API_URL	.env:2, .env.example:8, eas.json:16,27, lib/safetyApi.js:20	Configured — but read by code that is dead

lib/config.js is the module that actually performs the fetch, and it reads the variable that is never set. API_BASE therefore always resolves to the http://localhost:8000 fallback:

```
// lib/config.js:10-15
const RAW_BASE = process.env.EXPO_PUBLIC_SAFETY_API; // wrong name
... : 'http://localhost:8000'; // always taken
```

Consequence. On a physical device or APK, localhost refers to the phone itself. Report submission and journey saving **silently fail** and fall through to the offline queue — the user sees a "saved on your device" message and the data is never sent to the server.

Fix: change lib/config.js:10 to read EXPO_PUBLIC_SAFETY_API_URL to match .env and eas.json.

3. Working Fine

These areas were reviewed in depth and found to be correctly implemented, robustly handled, and honest in their user-facing claims.

AREA	ASSESSMENT
Splash & onboarding	Six-step flow with role-aware final routing, persistent <code>hasCompletedOnboarding</code> flag, animated slide/fade transitions, Android hardware-back support, full light and dark theme coverage. Verified end-to-end in a live browser session including the Skip path, relaunch behaviour and both citizen and government completions.
Emergency contacts	Full CRUD with add, edit, delete, duplicate-phone detection, length validation, delete confirmation and a proper empty state. Shares a single store with the SOS module — a contact added during onboarding was confirmed to appear on the Home screen as " <i>1 emergency contact ready</i> ".
SOS flow	Cancellable 3-second countdown, mandatory-contact gate, session persistence and restore, and correctly scoped background tracking. Notably, the copy is accurate: the app <i>prepares</i> an alert and the user chooses to send it — no false claim of automatic emergency dispatch.
Background location	A textbook-correct implementation: <code>defineTask</code> at module scope, try/catch guard against duplicate registration, foreground-to-background permission escalation, idempotency check via <code>hasStartedLocationUpdatesAsync</code> , foreground-service notification with <code>killServiceOnDestroy: false</code> , and matching Android manifest permissions.
Report offline queue	The best-implemented data path in the project: optimistic local insert, POST to server, swap of the optimistic record for the server record on success, and a durable pending queue on failure with an honest "saved on your device" message.
Routing & geocoding	Genuinely real. Four-tier routing fallback chain (Mapbox → OSRM direct → Valhalla → CORS proxy), Nominatim search with 600 ms debounce and a Tamil Nadu viewbox, plus real Leaflet maps in both native-WebView and web variants.
Journey navigation UI	Nearest-point-on-polyline matching, live ETA and remaining-distance recomputation, per-segment risk detection, animated bottom drawer, and star rating wired to the backend rating endpoint.
Storage abstraction	<code>lib/storage.js</code> cleanly abstracts AsyncStorage and <code>localStorage</code> behind a single API and is used correctly by preferences, localReports, journeyApi, emergencyContacts and onboarding.
Theme system	Two complete palettes provided through React context, consumed consistently across every screen. No <code>TODO</code> , <code>FIXME</code> or "coming soon" markers exist anywhere in the codebase.

4. Implemented but Partial

4.1 Safety score — real engine, unreachable client

HIGH

The backend scoring engine is genuinely real: Overpass street-lamp, police and hospital queries combined with Open-Meteo weather, aggregated through eight weighted factors that correctly sum to 1.00. The problem is entirely on the client side (defect 2.1).

There are now **four independent fallback tables** in the codebase, which is a maintenance hazard in itself:

LOCATION	VALUES	REACHABLE?
<code>safetyApi.js:111</code>	68 day / 82 night	Yes — always
<code>index.jsx:61</code>	82 day / 94 night	No — dead code
<code>safetyScore.jsx:19</code>	88 default, "Safe Zone"	Props only
<code>routes.jsx:105</code>	80, "LOW"	No — dead code

4.2 Report submission — persists, but never geo-tagged

HIGH

The submission path works, but the payload omits coordinates entirely:

```
// app/(tabs)/report.jsx:38-43
// no lat / lng anywhere, and useLocation() is never called
const result = await addReport({ category, location_label, details, image_uri });
```

- The backend's radius filter can therefore **never geo-query** — all reports are effectively unlocated
- The selected photo URI is **discarded**; only a `has_photo` boolean is transmitted, and no upload exists
- The success message states the report is *"now live on the community safety map"* — **no such map exists anywhere in the application**
- Server-side storage is an in-memory list, so all reports are lost on backend restart

4.3 Journey tracking — foreground only

HIGH

The journey screen uses `useLiveLocation()`, which is built on `watchPositionAsync` — foreground only. **Tracking stops the moment the app is backgrounded or the user leaves the screen.** The correctly-built background task is only ever started by the SOS module.

- The **Share button is a complete no-op on iOS and Android** — it is guarded by `Platform.OS === 'web'` and only calls `navigator.clipboard`
- Live-tracking errors are destructured but never surfaced, so a failure is invisible to the user

4.4 Profile & preferences — device-local only

MEDIUM

- All edits persist to local storage; **nothing syncs to any account or server**
- Avatar selection stores a local `file://` cache URI into a JSON profile — **the avatar breaks after app restart** on both Android and iOS
- Only `preciseLocation` is actually consumed by the codebase. `anonymousReports` has no effect, and `shareLocationInSOSOnly` is **never read at all**

4.5 Government dashboard — structurally empty

HIGH

The dashboard reads only the synchronous local cache and **never calls** `refreshReports()`. The root navigation gate bars government accounts from the tabs, and the tabs are the only place that fetch runs. The result: **on a fresh device the dashboard is permanently empty.**

It also contains a genuinely dead control — a button with no handler:

```
// app/gov.jsx:90
<Pressable style={s.reviewBtn}>
  <CheckCircle2 /> <Text>Mark reviewed</Text>
</Pressable>
// ← no onPress; nothing happens when tapped
```

A `RefreshControl` is also attached to a non-scrollable list nested inside a `ScrollView`, making pull-to-refresh unreachable.

4.6 Edit-profile location capture — stale coordinate race

MEDIUM

The current-location helper fires the request without awaiting it, then writes whatever value is *currently* held — possibly a stale fix from a previous visit. When the fresh fix arrives the loading flag has already been cleared, so the effect early-returns. The user silently saves **the previous coordinates** as their Home or Work location.

6. Not Yet Started

6.1 Real authentication

NOT STARTED

Entirely mocked. There is no Supabase client, no backend auth, and **no network call of any kind** in the auth module. It relies on its own `localStorage` helpers instead of the project's `lib/storage.js`, so:

- **Session never persists on Android or iOS** — the user is signed out on every launch
- `signUp` destructures `{ email, fullName, accountType, agency }` and **silently discards the password**; any string is accepted
- Plaintext `===` comparison against module constants; no hashing, no strength rules, no reset, no email verification
- `account_type` is a self-selected dropdown — **anyone can register as a government officer** and reach the government dashboard. There is no server-side role check

6.2 Database persistence (backend)

NOT STARTED

There is **no database and no file I/O of any kind** in `safety-engine`. Reports and journeys are held in module-level in-memory lists and dicts, capped at 500 records. **All data is lost on restart** and is not shared between worker processes. The source files acknowledge this honestly in their docstrings.

6.3 Backend security controls

NOT STARTED

- **No authentication, no rate limiting** on any of the 10 routes
- One `/safety/analyze` call fans out to roughly **10 concurrent POSTs to the free public Overpass instance**, which is a genuine abuse risk for third-party infrastructure
- CORS is fully open (`allow_origins=["*"]`), so any website a user visits can read and write the API
- No caching whatsoever — every request re-queries Overpass
- No configuration mechanism: `os.environ` appears **zero times** in the backend; every URL, weight, timeout and radius is a hardcoded constant

6.4 Automated tests

NOT STARTED

There are **no tests of any kind** — no Python tests and no JavaScript test runner configured. The `test_route.js` and `test_headers.js` files at the repository root are dead scripts targeting a Node backend that no longer exists; both are explicitly excluded by `.easignore` yet still committed.

6.5 Brand assets

NOT STARTED

There is **no real app icon in the repository**. Both `assets/images/icon.png` and the generated `android/.../splashscreen_logo.png` are the stock Bolt placeholder grid image — verified by inspection. The only genuine brand mark in the project is the shield glyph drawn in code.

6.6 Degraded-data signalling

NOT STARTED

This is the most consequential design gap for a *safety* product. When Overpass is unreachable, `/safety/spot` returns HTTP 200 with a confident-looking score computed from **all-zero data** — roughly **42, labelled "Low Safety — Stay Alert"** at night — with no degraded flag. A user cannot distinguish a genuinely risky street from a dead API. Separately, a total weather outage is scored as *perfect* weather, because the failure handler returns neutral defaults that happen to be optimal values.

7. Additional Defects

7.1 Runtime crashes — captured in the live console

Both were reproduced during live testing. Neither has an error boundary, so both produce a white screen.

Leaflet crash on incomplete coordinates

CRASH

```
Error: Invalid LatLng object: (undefined, undefined)
  at new LatLng ...
  at createMarker ...
Call Stack:
  RouteMap (components\RouteMap.web.jsx:42:19)
  RoutesScreen (app\tabs\routes.jsx:264:11)
```

The guard checks that `source` is truthy but never validates that `source.lat` and `source.lng` exist.

Double-fault in reverse geocoding

CRASH

```
Error: Reverse Geocode Error: TypeError:
  Cannot read properties of undefined (reading 'toFixed')
  at location.js:501:26 // primary failure
  at location.js:506:24 // the .catch() fallback fails identically
```

The recovery handler contains the same `lat.toFixed(4)` call that just threw, so it throws again inside the `catch` and the promise never resolves with a fallback.

7.2 Undefined theme keys

Five colour keys are referenced in the app but are not defined in `lib/theme.jsx`. They resolve to `undefined` and render as transparent or invisible:

KEY	USES	APPEARS IN	VISIBLE RESULT
<code>colors.pageBg</code>	2	<code>journey.jsx</code>	Transparent background
<code>colors.green</code>	2	<code>SOSActiveScreen.jsx</code>	Invisible text
<code>colors.greenSoft</code>	1	<code>SOSActiveScreen.jsx</code>	Transparent fill
<code>colors.amber</code>	2	<code>SOSActiveScreen.jsx</code>	Invisible text
<code>colors.amberSoft</code>	1	<code>SOSActiveScreen.jsx</code>	Transparent fill

7.3 Stale state and duplicate handlers

- `routes.jsx:253` — changing travel mode clears `route` but not `allRoutes` or `safetyData`, so the "Start Journey" button stays visible holding a previous-mode route
- `routes.jsx:436` — both `onPressIn` and `onPress` invoke the same handler, so every tap fires twice
- `useLocation()` is instantiated three times independently, each holding its own state and firing its own GPS fix and permission prompt
- Duplicate `<StatusBar>` instances are mounted in both the root layout and the theme provider
- Two `FlatList` components with `scrollEnabled={false}` are nested inside `ScrollView` parents, which React Native explicitly warns against
- Four map components each carry their own hardcoded palette that does not match the application theme

7.4 Repository hygiene

ISSUE	DETAIL
Android prebuild committed	37 native files are tracked but <code>android/</code> is not in <code>.gitignore</code> , so native permission changes require a manual prebuild sync and can silently drift from <code>app.json</code> .
Placeholder credentials shipped	<code>eas.json</code> contains <code>"pk.your-token-here"</code> , so the Mapbox routing path is always skipped and silently falls through.
Developer LAN IP committed	<code>eas.json:16,27</code> hardcode <code>http://192.168.0.6:8000</code> , tied to one machine.
Foreign contact identity	<code>lib/location.js</code> sends <code>&email=contact@safetransit.com</code> to Nominatim in three places — a different project's identity, and a violation of the Nominatim usage policy.
App scheme not renamed	The URL scheme is <code>"myapp"</code> in both <code>app.json</code> and the Android manifest.
Dead files at root	<code>express_route_patch.js</code> is a superseded Express stub for a service the backend comments describe as "nonexistent".
Lint config untracked	<code>eslint.config.js</code> is untracked and depends on packages absent from <code>package.json</code> .

8. Recommended Order of Work

Start here — two lines restore three features

HIGHEST LEVERAGE

- 1 **Add the missing import in `lib/safetyApi.js`** . This single line activates the real safety score, the safe-hub data and the entire route-planning flow.
- 2 **Align the environment variable in `lib/config.js`** . This restores report submission and journey saving on physical devices.
- 3 **Fix the two crash paths in `RouteMap.web.jsx:42` and `lib/location.js:501,506`** , and add a top-level error boundary.
- 4 **Add the five missing theme keys (`pageBg` , `green` , `greenSoft` , `amber` , `amberSoft`)** or replace their usages with the existing `teal` and `orange` .
- 5 **Call `refreshReports()` on the government dashboard** mount, and either implement or remove the "Mark reviewed" button.

Then, in priority order: implement real authentication → introduce a database → add backend authentication, rate limiting and caching → add degraded-data flags so the app never presents invented data as real → promote the journey tracker to the background task → add a test suite → replace the placeholder brand assets.

Severity summary

CATEGORY	COUNT	NOTES
CRITICAL	2	Missing import and env-var mismatch — both silent, both fixable in ~2 lines
CRASH	2	Leaflet <code>Invalid LatLng</code> and the double-faulting geocode fallback; both white-screen a session
HIGH	5	Report geo-tagging, journey background tracking, government dashboard, no auth, no tests
MEDIUM	3	Profile persistence, stale coordinate race, missing theme keys
VERIFIED SOUND	9	Onboarding, contacts, SOS, background location, offline queue, routing, theme, storage, journey UI